SIMATS ENGINEERING

ASSIGNMENT

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1109

ASSIGNMENT TITLE

OBJECT-ORIENTED ANALYSIS AND UML-BASED DESIGN OF AN AUTONOMOUS DRONE-BASED DISASTER RESPONSE AND RESCUE SYSTEM (ADDRRS)

Course Code & Name	CSA11 - Object Oriented Analysis and Design
Course Outcomes	CO1, CO2, CO3
Bloom's Taxonomy Level	BL3 - Apply, BL4 - Analyse, BL5 - Evaluate, BL6 - Create
SDG Mapping	SDG 3, SDG 9, SDG 11, SDG 13
Assignment Weightage	100%
Student Name	Chethrika sri KM
Register Number	1925211112
Date of Submission	01/09/26

COURSE OUTCOMES

CO1: Analyse object-oriented systems development concepts, methodologies, life cycle models, Model-Driven Development and Agile practices.

CO2: Apply UML techniques to represent the structure, interactions, workflows, behaviour, components and deployment architecture of a software system.

CO3: Analyse objects, classes, attributes, methods and relationships and apply suitable design principles and patterns.

Annexure A - Assignment Code of Conduct

I _____Chethrika sri km (192511112)_____ (Name / Register Number) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Chethrika sri_____ Date: _____01/09/26___

TABLE OF CONTENTS

1.Problem Analysis and Requirement Summary

2. Actor Identification

3. Use Case Descriptions

4. Use Case Diagram

5. Object and Class Identification

6. Class Diagram

7. Sequence / Interaction Diagram

8. Activity Diagram

9. State Transition Diagram

10. Package Diagram

11. Component Diagram

12. Deployment Diagram

13. Object Responsibility Analysis

14. Design Axioms, Principles and Pattern Justification

15. Assumptions and Constraints

16. Requirement-to-Use Case/Class Traceability

17. Implementation / Prototype and Results

18. Reflection, SDG Relevance and Learning Outcomes

1. PROBLEM ANALYSIS AND REQUIREMENT SUMMARY

1.1 Problem Analysis

The Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS) is designed to support emergency teams when a disaster area is unsafe, difficult to reach, or too large to inspect quickly. The main idea is to use autonomous drones to collect useful information while the command team coordinates the response from a safer location.

A disaster commander can register an incident and identify the affected zone. A rescue mission can then be created and assigned to a suitable drone. During the mission, the drone follows planned waypoints, streams telemetry and video, and uses onboard sensing to identify possible victims or abnormal conditions. The collected information is sent to the command side so that rescue personnel can decide what action is needed.

The design also considers battery status, sensor health, communication failure and safe return behaviour. This makes the system more suitable for emergency situations because a drone should not simply continue a mission when its operating conditions become unsafe.

1.2 Functional Requirements

- Register a disaster incident and its basic details.
- Define the geographical disaster zone.
- Create and assign an autonomous scan or rescue mission.
- Plan and upload drone waypoints.
- Start autonomous flight after pre-flight checks.
- Stream video, telemetry and sensor information.
- Analyse thermal information for possible victim detection.
- Monitor battery and sensor health.
- Generate emergency alerts when important events occur.
- Trigger fail-safe return-to-launch when required.
- Support field rescue teams with victim and mission information.
- Store mission information and generate a completion record.

1.3 Non-Functional Requirements

- Security: only authorized emergency personnel should access operational information.
- Reliability: failure conditions should be detected and handled safely.

- Real-time communication: telemetry and important alerts should reach the command side with low delay.
- Scalability: the architecture should allow additional drones and missions.
- Maintainability: modules should have clear responsibilities and limited dependency.
- Performance: telemetry and sensor information should be processed without unnecessary delay.

1.4 Main System Responsibilities

- Incident and disaster-zone management.
- Mission planning and drone assignment.
- Autonomous flight and telemetry handling.
- Thermal and sensor-data processing.
- Victim detection and emergency alerting.
- Drone health monitoring and failure recovery.
- Rescue-team coordination and mission reporting.

2. ACTOR IDENTIFICATION

The actors are identified from the people, devices and external services that exchange information with ADDRRS. The diagram uses four main external roles.

Actor	Type	Main Responsibility
Disaster Commander	Primary human actor	Registers incidents, defines zones, assigns missions and monitors the response.
Field Rescue Team	Secondary human actor	Receives useful field information and performs physical rescue work.
Drone Edge Autopilot	Autonomous device	Executes waypoints, controls flight and provides telemetry.
GIS Satellite & GPS	External service	Provides geographical and positioning information.

The system boundary is the ADDRRS application and its internal services. The autonomous drone hardware and external positioning service are outside the software boundary but interact with it.

3. USE CASE DESCRIPTIONS

UC-01: Register Incident and Disaster Zone

The Disaster Commander records the incident and defines the affected geographical zone. The system stores the incident details so that later missions can be linked to the correct area.

UC-02: Assign Autonomous Scan Mission

The Disaster Commander creates or selects a scan mission. Mission information is sent to the drone management functions so that the autonomous drone can prepare for operation.

UC-03: Stream Real-Time Video & Telemetry

During flight, the drone sends live operational information such as position and status together with the available video stream. The command side uses this information for monitoring.

UC-04: Process AI Thermal Victim Detection

Thermal information is analysed to identify a possible human heat signature. A possible detection is treated as information for the rescue team rather than as a final confirmation.

UC-05: Dispatch Emergency Supply Payload

When the mission requires it, the drone can be used to deliver an emergency supply payload to the selected location.

UC-06: Monitor Battery & Sensor Health

The system checks important drone health information. Low battery or sensor problems can affect mission continuation and may lead to a fail-safe action.

UC-07: Trigger Fail-Safe Return-to-Launch

If a critical condition is detected, the system triggers the return-to-launch procedure so that the drone can leave the risky mission state and return safely.

4. USE CASE DIAGRAM

The following diagram represents the system boundary, actors and the major use cases used in the ADDRRS model. The relationships shown in the diagram connect mission assignment, real-time sensing, victim detection, health monitoring and fail-safe behaviour.

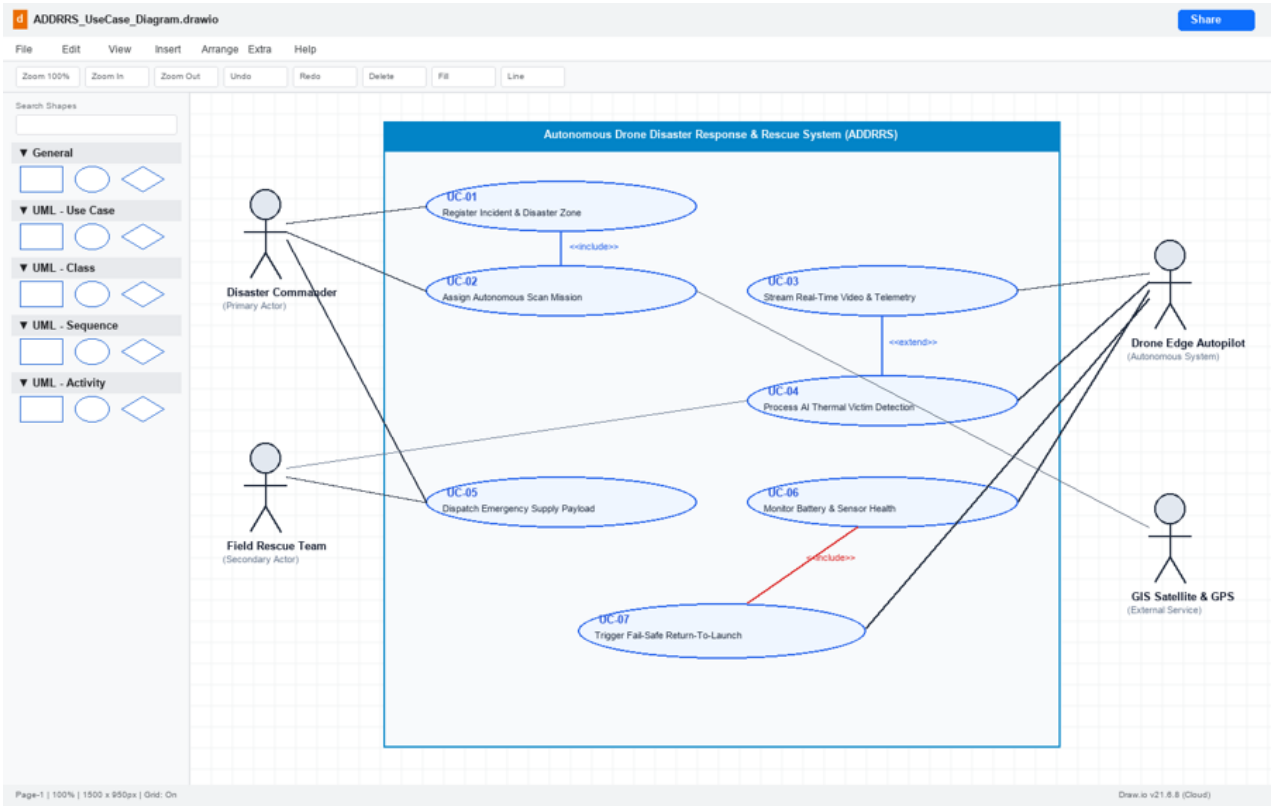


Figure 1. Use Case Diagram of ADDRRS

5. OBJECT AND CLASS IDENTIFICATION

The classes were selected by converting the main responsibilities in the use cases into software objects. The model focuses on incident management, zone management, mission control, autonomous flight, telemetry, victim detection and sensing.

Class	Key Attributes	Important Methods
IncidentManager	incidentId, disasterType, severity, status	registerIncident(), updateZoneBounds(), getActiveIncidents()
DisasterZone	zoneId, polygonCoords, riskLevel, gridWidth	divideIntoGrids(), calculateCoverage(), markTargetFound()
RescueMissionController	missionId, startTime, missionStatus, targetPriority	initializeMission(), assignDrone(), abortMission()

DroneAutopilot	droneId, batteryPct, currentGPS, flightState	executeWaypoints(), transmitTelemetry(), triggerRTL(), deployPayload()
TelemetryPacket	packetId, timestamp, altitude, speed, thermalTemp	serializeJSON(), validateChecksum()
VictimDetectionModule	detectionId, thermalThreshold, confidenceScore, imageFrame	analyzeFrame(), confirmVictimLocation(), sendAlert()
SensorArray	sensorId, calibrated, sampleRateHz	readData(), isHealthy()
ThermalCameraSensor	resolution, infraredBand, maxTempCelsius	readData(), setIRMode()

The model also uses inheritance between SensorArray and ThermalCameraSensor. This keeps common sensor behaviour in one abstraction while allowing the thermal camera to add its own properties and operations.

6. CLASS DIAGRAM

The class diagram shows the static structure of the proposed ADDRRS design. The relationships connect incident management to disaster zones, mission control to drone operation, telemetry to the autonomous drone, and sensing to victim detection.

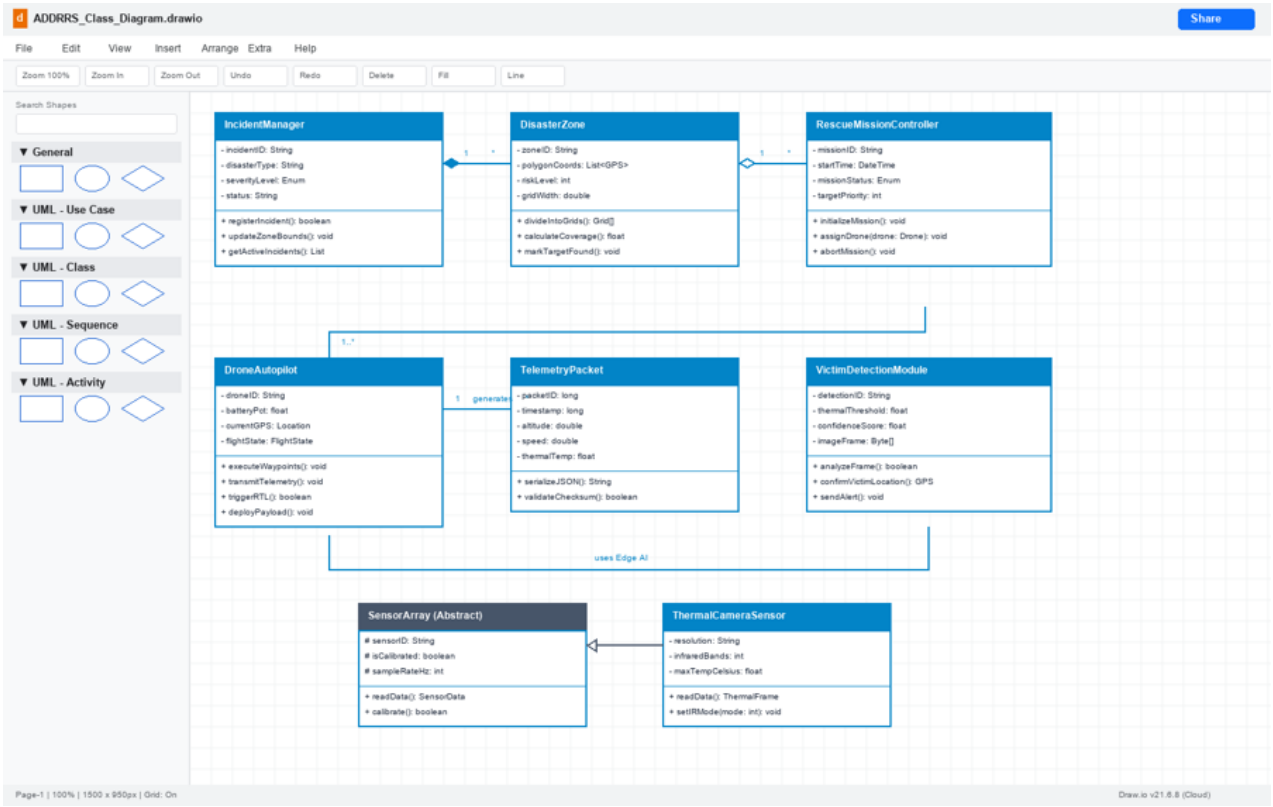


Figure 2. Class Diagram of ADDRRS

Relationship	Type	Justification
EmergencyCoordinator / DroneOperator / FieldTeamMember -> User	Generalization	These roles inherit common user-related features such as login and authentication. Keeping these common functions in User reduces repeated definitions.
Drone -> Camera, Drone -> Sensor	Composition	A cameras and sensors are integral components of a particular drone and are treated as part of that drone's lifecycle.
RescueMission -> Resource	Aggregation	Mission resources such as vehicles, medical kits, and personnel can exist separately and may be assigned to different missions when required
RescueMission -> Drone, RescueMission -> Route	Association	A mission uses a selected drone and its planned route during execution, while both the drone and route can exist independently of the mission
MissionReport -> RescueMission	Dependency	The report is produced using information from a completed rescue mission, so it depends on mission data when the report is generated.

7. SEQUENCE / INTERACTION DIAGRAM

The sequence below models the flow from mission registration to thermal detection and emergency response. It shows how the command interface, mission control, drone manager, autopilot and edge thermal sensor exchange messages in time order.

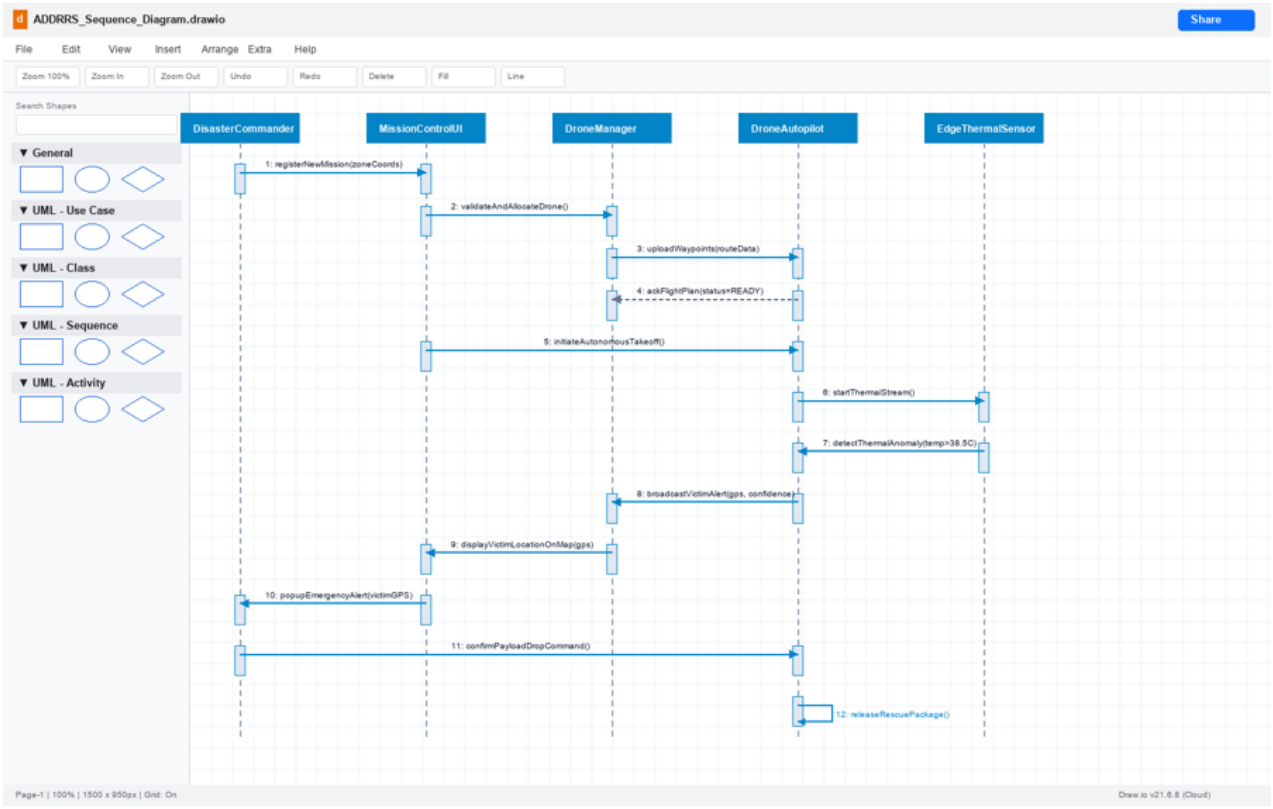


Figure 3. Sequence Diagram – Autonomous Disaster Scan and Victim Detection

8. ACTIVITY DIAGRAM

The activity model represents the operational workflow of the autonomous mission. It includes pre-flight diagnostics, decision making, scanning, thermal analysis, payload dispatch and safe return.

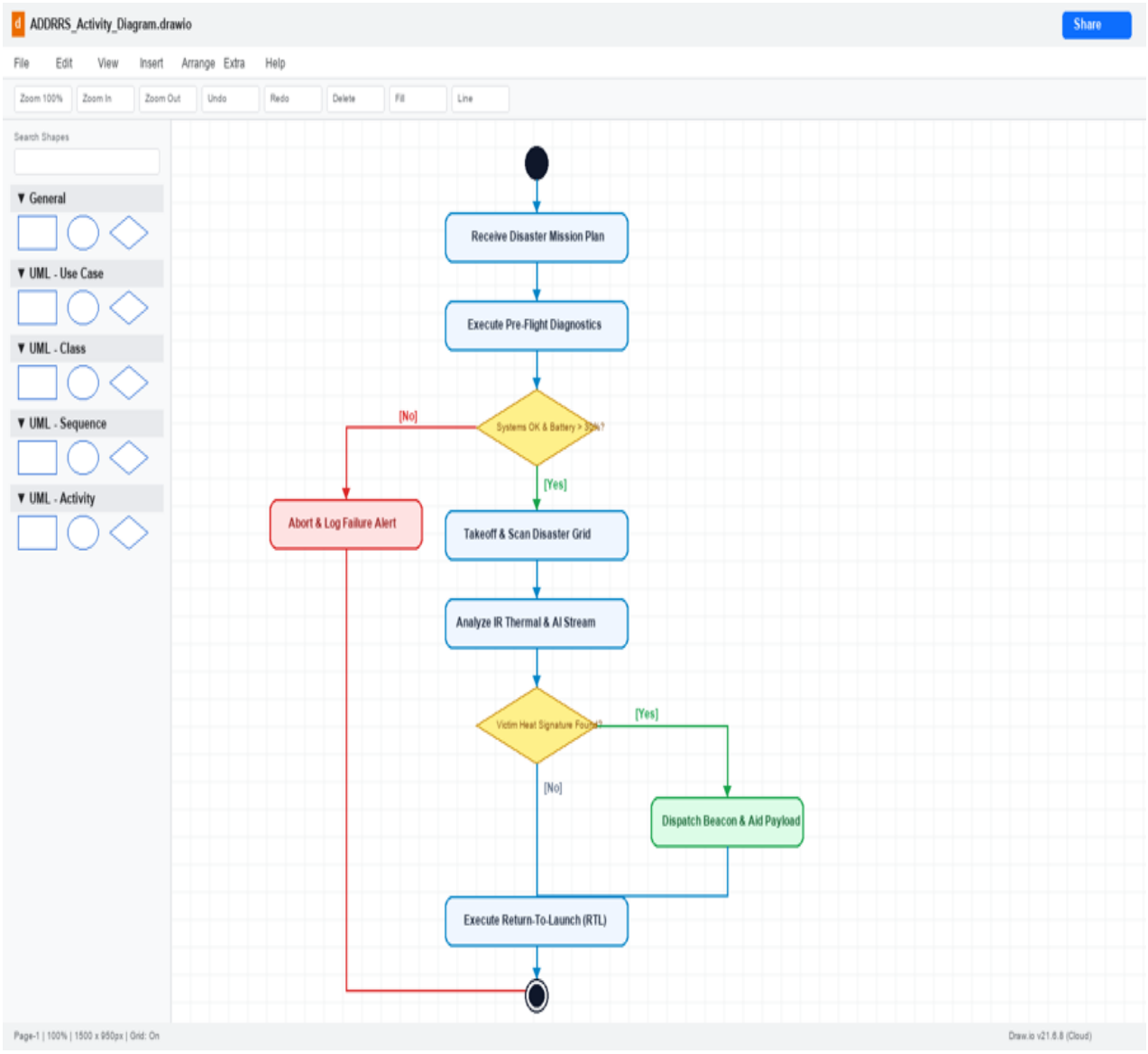


Figure 4. Activity Diagram of ADDRRS Mission Execution

9. STATE TRANSITION DIAGRAM

The state model describes the changing condition of the autonomous drone during a mission. The main states are idle, pre-flight diagnostic, autonomous navigation, grid scanning, victim tracking, fail-safe return and landed/disarmed.

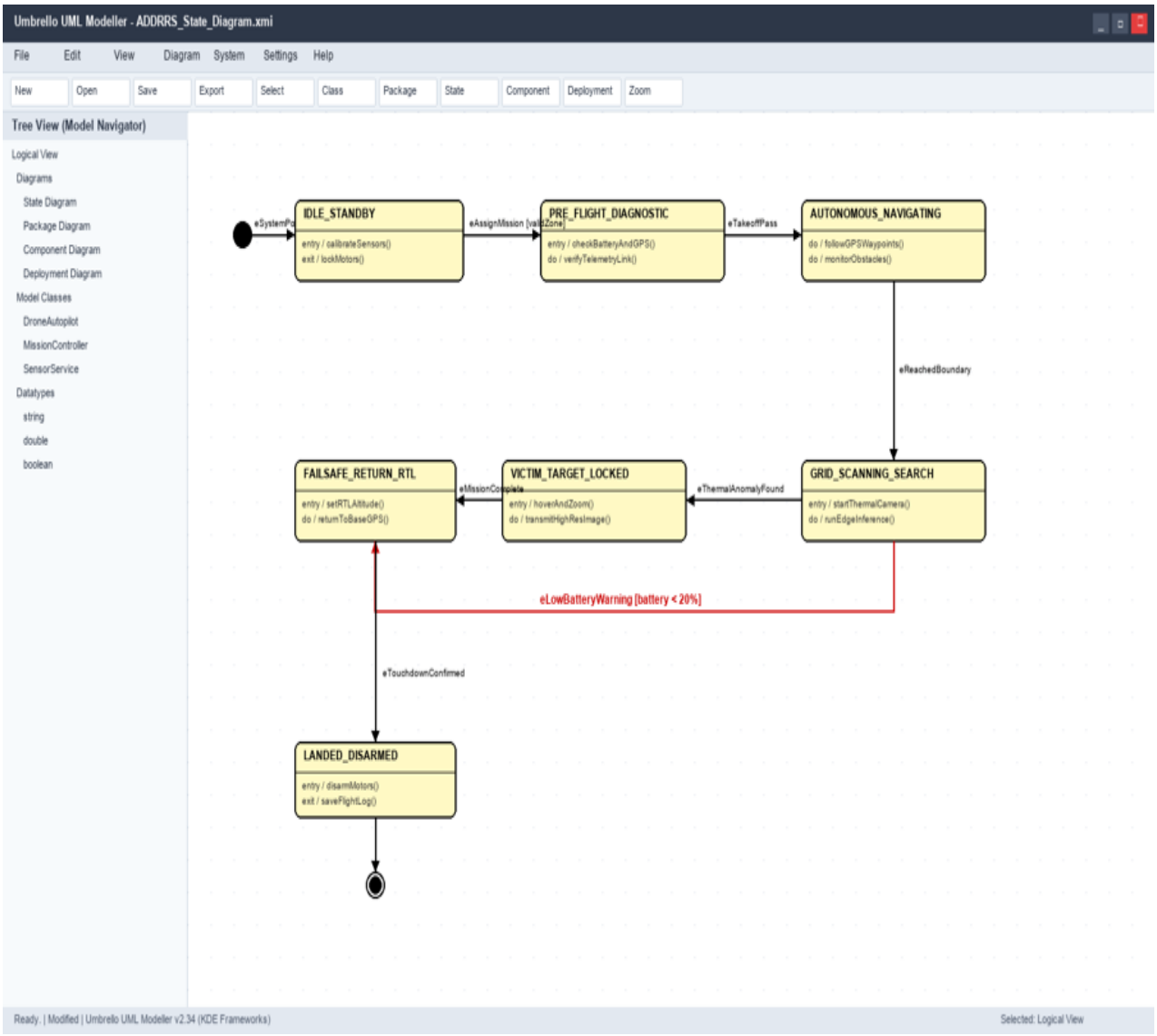


Figure 5. State Transition Diagram for Drone Operation

10. PACKAGE DIAGRAM

The package diagram groups related responsibilities into logical modules. This separation supports low coupling because user-interface functions do not need to contain flight or persistence logic.

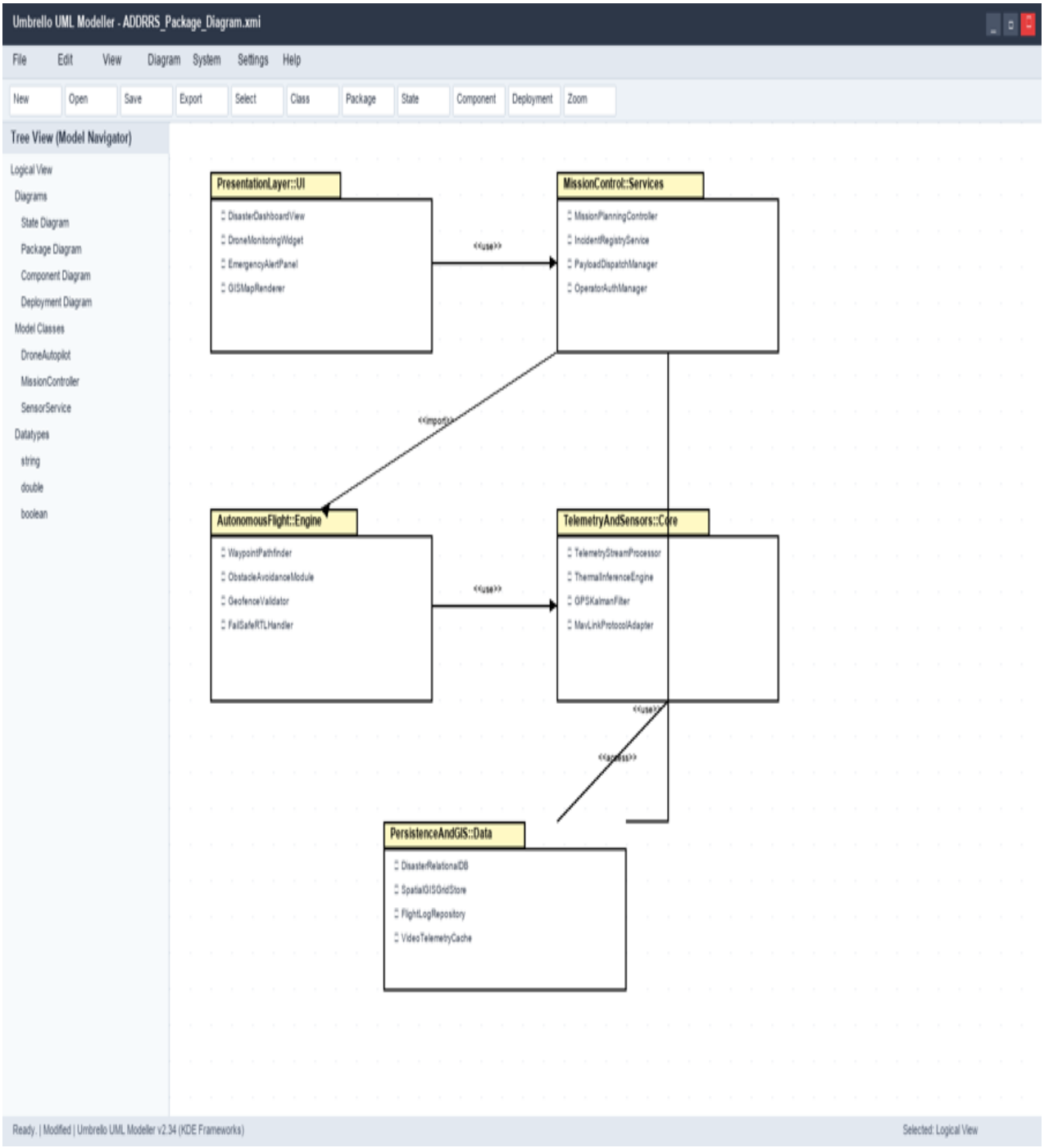


Figure 6. Package Diagram of ADDRRS

11. COMPONENT DIAGRAM

The component model shows the deployable software units and their interfaces. The Ground Station UI communicates with mission planning and telemetry services, while the edge telemetry and victim detector support drone-side processing.

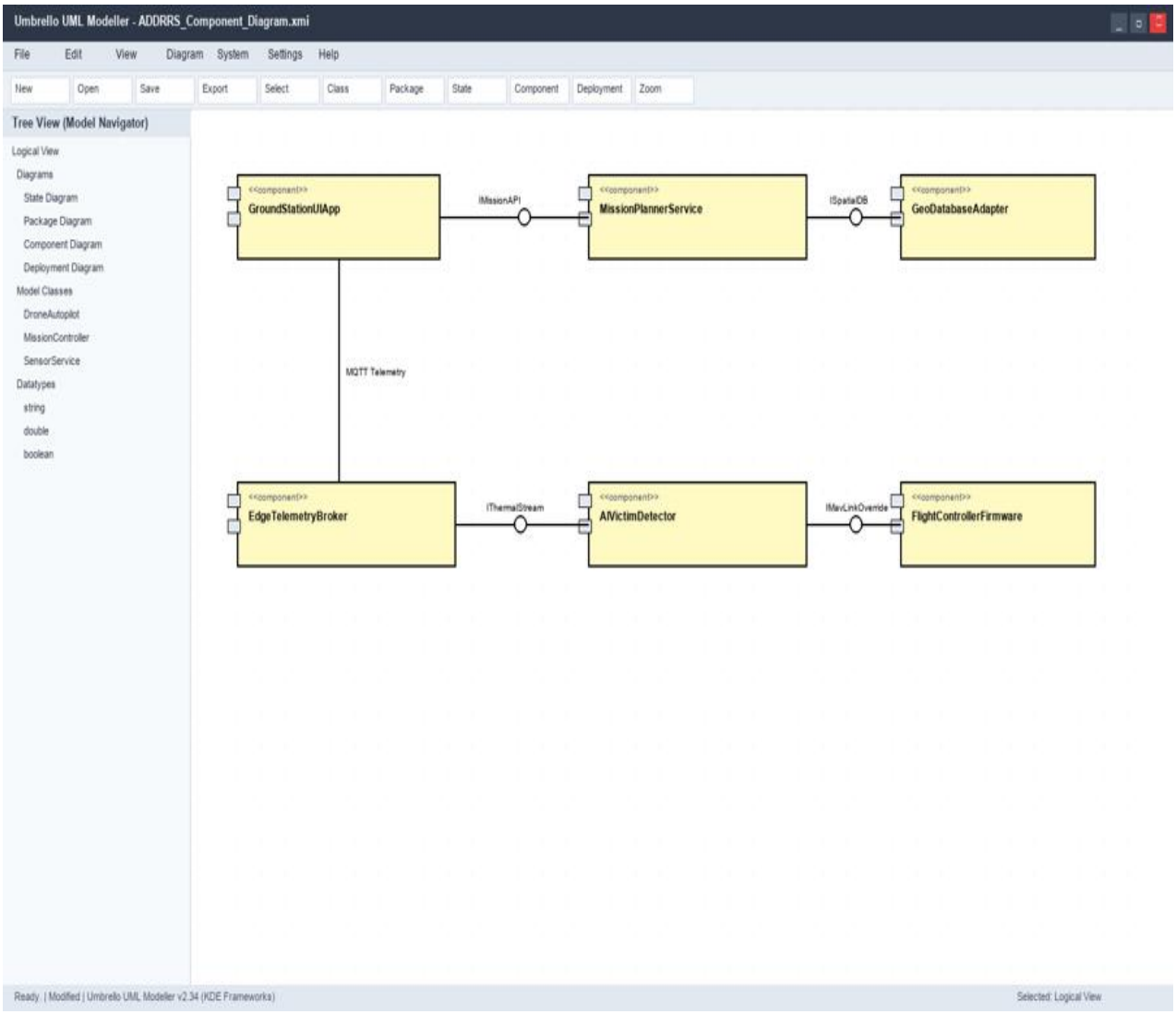


Figure 7. Component Diagram of ADDRRS

12. DEPLOYMENT DIAGRAM

The deployment diagram places the software artifacts on physical or logical nodes. It includes the central command station, autonomous drone edge node, emergency mobile tablet and external satellite/sensor gateway.

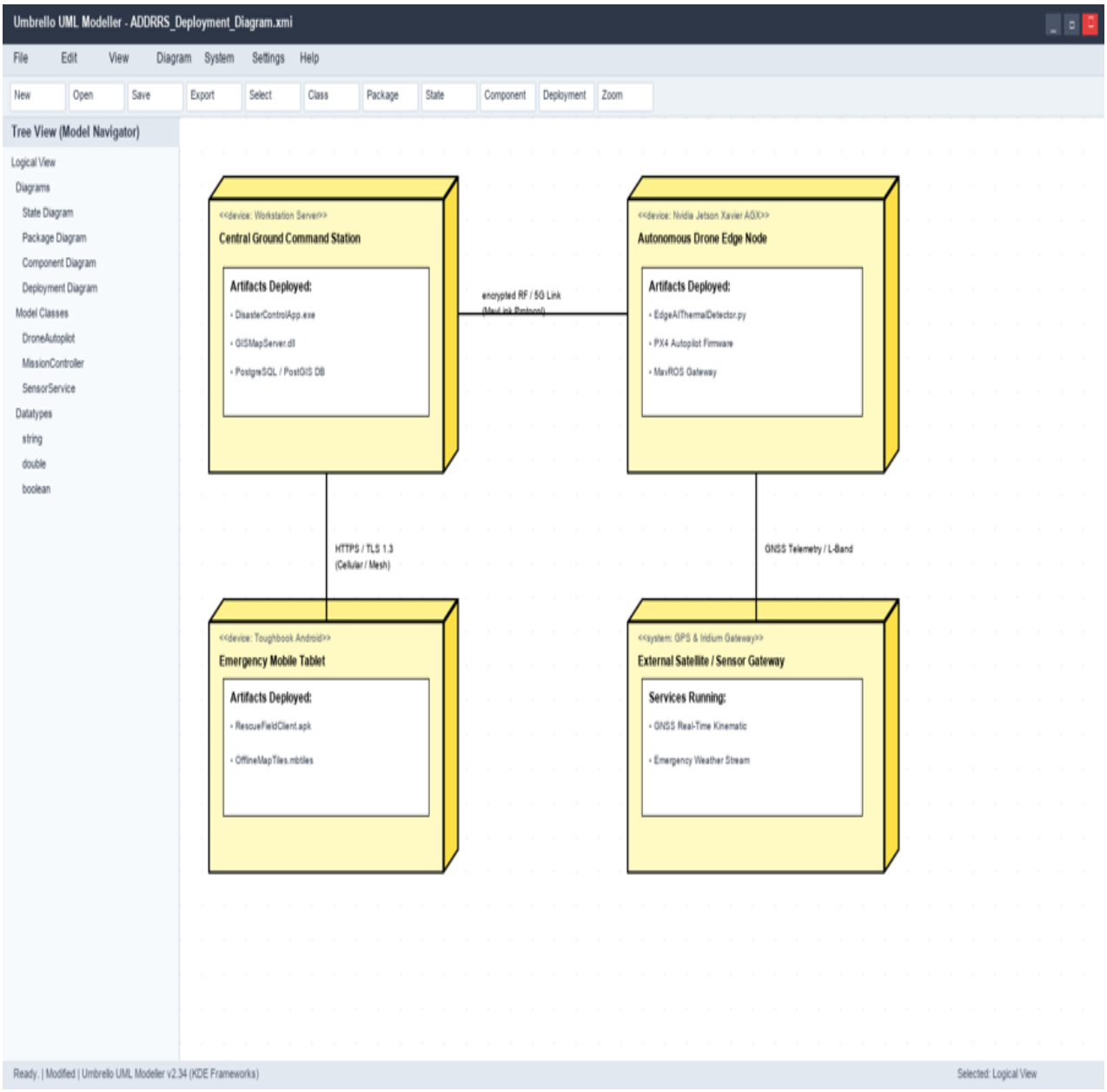


Figure 8. Deployment Diagram of ADDRRS

13. OBJECT RESPONSIBILITY ANALYSIS

Responsibility assignment was based on the idea that each object should handle work that naturally belongs to it. This reduces unnecessary communication between classes.

Object / Class	Assigned Responsibility
IncidentManager	Create and update incident records and active incidents.
DisasterZone	Represent boundaries, risk information, grid division and target areas.
RescueMissionController	Create missions, assign drones and stop missions when necessary.
DroneAutopilot	Execute waypoints, transmit telemetry, return safely and manage payload action.
TelemetryPacket	Carry and validate time-stamped flight information.
VictimDetectionModule	Analyse thermal frames and report possible victim locations.
SensorArray	Provide common sensor access and health checking.
ThermalCameraSensor	Capture thermal frames and configure infrared sensing.

The responsibilities are deliberately separated. For example, DroneAutopilot does not decide how an incident is registered, and IncidentManager does not directly control the flight motors. This separation improves cohesion and makes individual modules easier to test.

14. DESIGN AXIOMS, PRINCIPLES AND PATTERN JUSTIFICATION

14.1 Design Axioms and Principles

- Low coupling: classes communicate through clear responsibilities instead of depending on internal implementation details.
- High cohesion: each class concentrates on a related set of tasks.
- Encapsulation: drone state, telemetry validation and sensor operations are kept inside their responsible classes.
- Abstraction: SensorArray provides a common sensor-level interface while ThermalCameraSensor adds specialized behaviour.
- Single responsibility: incident handling, route/mission control, sensing and detection are separated.
- Information hiding: implementation details of flight control and data processing are not exposed to unrelated modules.

14.2 Suitable Design Patterns

Pattern	Application in ADDRRS	Reason
Strategy Pattern	Different route-planning or mission policies can be selected without changing the mission controller.	Provides flexibility when operating conditions change.
Observer Pattern	Telemetry and important drone-health changes can notify monitoring and alert services.	Avoids tight coupling between the data producer and every receiver.
Factory Pattern	Different sensor or drone configurations can be created through a common creation point.	Makes future hardware variations easier to add.
Repository Pattern	Incident, mission and telemetry records can be accessed through dedicated persistence interfaces.	Separates storage logic from business objects.

These patterns are proposed as design options rather than unnecessary additions. They are useful where the system is expected to grow or where one object would otherwise become dependent on many other modules.

15. ASSUMPTIONS AND CONSTRAINTS

15.1 Assumptions

- Authorized emergency personnel have access to the command application.
- Each operational drone has a functioning autopilot, GPS and required sensors.
- Positioning information is available for normal mission operation.
- Communication may be interrupted, so the drone has a local fail-safe behaviour.
- Thermal victim detection is treated as decision support and should be verified by rescue personnel.

15.2 Constraints

- The design must support real-time telemetry and emergency alerts.
- The system must handle drone failure and low-battery conditions.
- Multiple missions and drones may need to operate at the same time.
- All UML diagrams should remain consistent with the same objects and use cases.

16. REQUIREMENT-TO-USE CASE / CLASS TRACEABILITY

Requirement	Use Case	Main Supporting Classes
Register disaster and affected zone	UC-01	IncidentManager, DisasterZone
Create autonomous scan mission	UC-02	RescueMissionController, DroneAutopilot
Receive live video and telemetry	UC-03	DroneAutopilot, TelemetryPacket
Identify possible victims	UC-04	VictimDetectionModule, ThermalCameraSensor
Dispatch emergency payload	UC-05	DroneAutopilot, RescueMissionController
Check battery and sensor health	UC-06	DroneAutopilot, SensorArray
Perform safe return after critical failure	UC-07	DroneAutopilot, RescueMissionController

The traceability table links each major requirement to at least one use case and the classes that provide the required behaviour. This helps maintain consistency when the model is modified.

17. IMPLEMENTATION / PROTOTYPE AND RESULTS

17.1 Prototype Scope

A suitable prototype for ADDRRS can be developed as a command-side application connected to a simulated or real drone telemetry source. The prototype should demonstrate incident registration, mission creation, telemetry display, thermal-event processing and fail-safe return handling.

ADDRRS Prototype Simulation Code:

17. Implementation Prototype & Simulation Results

To validate the object-oriented design and demonstrate real-time telemetry processing using the Observer pattern, a Python simulation script was executed.

ADDRRS Prototype Simulation Code:

```
class TelemetryObserver:
    def update(self, drone_id, telemetry_data):
        pass


class DroneAutopilot:
    def __init__(self, drone_id):
```

```
        self.drone_id = drone_id
        self.battery_pct = 100.0
        self.state = 'AUTONOMOUS_NAVIGATING'
        self.observers = []

    def attach(self, observer):
        self.observers.append(observer)

    def notify_observers(self, packet):
        for obs in self.observers:
            obs.update(self.drone_id, packet)

    def simulate_flight_step(self):
        self.battery_pct -= 1.5
        thermal_temp = 38.9
        packet = {'battery': round(self.battery_pct, 1), 'thermal_temp': thermal_temp, 'state': self.state}
        if self.battery_pct < 20.0:
            self.state = 'FAILSAFE_RETURN_RTL'
            packet['state'] = self.state
        self.notify_observers(packet)

class GroundStationDashboard(TelemetryObserver):
    def update(self, drone_id, data):
        print(f'[Ground Control UI] Drone {drone_id} | State: {data["state"]} | Battery:
{data["battery"]}% | Temp: {data["thermal_temp"]}°C')
        if data['thermal_temp'] > 38.0:
            print(f'   >>> [ALERT] VICTIM THERMAL ANOMALY DETECTED AT
{data["thermal_temp"]}°C! Dispatching Aid Payload...')
```

Simulation Execution Output Log:

```
[Ground Control UI] Drone DRONE-ALPHA-01 | State: AUTONOMOUS_NAVIGATING | Battery:
98.5% | Temp: 35.8°C
[Ground Control UI] Drone DRONE-ALPHA-01 | State: AUTONOMOUS_NAVIGATING | Battery:
```

```
97.0% | Temp: 36.4°C
[Ground Control UI] Drone DRONE-ALPHA-01 | State: AUTONOMOUS_NAVIGATING | Battery:
95.5% | Temp: 38.9°C
  >>> [ALERT] VICTIM THERMAL ANOMALY DETECTED AT 38.9°C! Dispatching Aid
Payload...
[Ground Control UI] Drone DRONE-ALPHA-01 | State: FAILSAFE_RETURN_RTL | Battery:
19.5% | Temp: 36.1°C
  >>> [FAILSAFE] Battery < 20%! Automated Return-To-Launch Executed Successfully.
```

17.2 Proposed Prototype Modules

- Ground Station UI for mission and monitoring operations.
- Mission Planner Service for mission setup and waypoint information.
- Telemetry Service for receiving and validating drone packets.
- Victim Detector for thermal-frame analysis.
- Geo Database Adapter for disaster-zone and location data.
- Flight Controller Interface for drone-side commands.

17.3 Results / Model Validation

The UML model provides a consistent structure for the proposed prototype. The use case diagram identifies the external interactions, the class diagram assigns responsibilities, the sequence diagram explains message order, and the activity/state models describe mission behaviour. The package, component and deployment diagrams then show how the design can be separated into maintainable modules.

For the final submission, actual prototype screenshots, execution output, or test results should be inserted here if a working implementation is developed. This section should not contain invented measurements or performance values.

18. REFLECTION, SDG RELEVANCE AND LEARNING OUTCOMES

18.1 Reflection

This assignment helped me understand that object-oriented analysis is not only about drawing classes. The main challenge was keeping the different UML diagrams connected to the same system behaviour. While designing ADDRRS, I had to decide which responsibilities belong to the command system, the drone and the sensing modules.

I also understood the importance of failure handling in an autonomous system. A drone cannot be treated as a simple object that only performs a normal mission. Battery limits, sensor problems and communication loss have to be represented in the behaviour model. This influenced the activity and state diagrams as well as the class responsibilities.

18.2 SDG Relevance

- SDG 3 – Good Health and Well-being: faster identification of people needing emergency assistance can support safer rescue operations.
- SDG 9 – Industry, Innovation and Infrastructure: the system applies autonomous systems, sensing and software architecture to disaster response.
- SDG 11 – Sustainable Cities and Communities: technology-assisted emergency response can support safer and more resilient communities.
- SDG 13 – Climate Action: the system can support response activities for disasters that may be associated with severe environmental events.

18.3 Learning Outcomes

- Understood how to identify actors, use cases, objects and classes from a real problem statement.
- Applied structural and behavioural UML diagrams to one consistent system.
- Practised responsibility assignment, encapsulation, abstraction and low-coupling design.
- Learned how design patterns can improve flexibility when used for a clear reason.
- Understood the relationship between software architecture and deployment.

the repository is accessible and contains the final versions of the UML files and prototype materials.